FINAL PROJECT REPORT: US Stock Market Analysis & Prediction

1. Project Objective

The goal of this project was to analyze real-world financial data from Yahoo Finance to identify market trends, categorize companies by their market capitalization, and build a predictive model to understand the relationship between trading volume, market cap, and stock price.

2. Data Collection Method

- **Source:** Yahoo Finance
- **Method:** Web Scraping (Phase 1)
- **Tools:** Python (BeautifulSoup / Selenium / Requests)
- **Raw Data:** A dataset containing 1,000 stock records with attributes including Ticker, Company Name, Price, Change, Volume, and Market Cap.

```python
import requests
import pandas as pd
import time

def scrape_yahoo_bulk(target_count=1000):
    all_stocks = []
    # Yahoo typically allows 100 items per request
    offset = 0
    count_per_page = 100

    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36'
    }

    while len(all_stocks) < target_count:
        # This URL targets the 'Most Active' stocks screener API
        url = f"https://query1.finance.yahoo.com/v1/finance/screener/predefined/saved?formatted=true&scrIds=most_actives&count={count_per_page}&offset={o

        try:
            response = requests.get(url, headers=headers)
            data = response.json()

            # Navigate the JSON structure
            quotes = data['finance']['result'][0]['quotes']

            if not quotes:
                break
```

```python
            for stock in quotes:
                all_stocks.append({
                    # CATEGORICAL COLUMNS
                    'Ticker': stock.get('symbol'),
                    'Company': stock.get('shortName'),
                    'Market': stock.get('market'),
                    'QuoteType': stock.get('quoteType'),

                    # NUMERICAL COLUMNS
                    'Price': stock.get('regularMarketPrice', {}).get('raw', 0),
                    'Change': stock.get('regularMarketChange', {}).get('raw', 0),
                    'Volume': stock.get('regularMarketVolume', {}).get('raw', 0),
                    'MarketCap': stock.get('marketCap', {}).get('raw', 0)
                })

            print(f"Collected {len(all_stocks)} rows...")
            offset += count_per_page
            time.sleep(1) # Be nice to their servers!

        except Exception as e:
            print(f"Reached end or encountered error: {e}")
            break

    # Convert to a DataFrame for analysis
    df = pd.DataFrame(all_stocks)
    return df

# Execute
final_df = scrape_yahoo_bulk(1000)

# Save to CSV
final_df.to_csv("yahoo_finance_1000_rows.csv", index=False)
print("Saved to yahoo_finance_1000_rows.csv")
```

3. Data Cleaning & Preprocessing

To ensure the data was decision-ready, the following steps were taken:

- **Handling Missing Values:** Dropped rows with null values in critical columns like Price and MarketCap.
- **Data Type Conversion:** Converted string-based financial figures (e.g., Market Cap) into numeric floats and integers for calculation.
- **Feature Engineering:** * Created a **Price Band** (Low, Mid, High) using quantiles.
    - Categorized companies into **Market Cap Sizes** (Small, Mid, Large, Mega).
    - Developed a **Volatility Index** to measure price stability.

```
[ ]: import pandas as pd
     import numpy as np
     import seaborn as sns
     import matplotlib.pyplot as plt
```

PHASE 2: CLEANING

```
[ ]: df = pd.read_csv('yahoo_finance.csv')
```

```
[ ]: df.head()
```

```
[ ]: # 1. Handle Missing Values & Duplicates
     df_clean = df.drop_duplicates().reset_index(drop=True)
     df_clean['PriceCategory'] = pd.cut(df_clean['Price'], bins=[0, 20, 100, 1000], labels=['Low', 'Medium', 'High'])
     df_clean.to_csv('cleaned_yahoo_finance.csv', index=False)
     print('cleaned_yahoo_finance.csv')
```

```
[ ]: # 2. Fix inconsistent formats
     # Ensure Price and MarketCap are numeric
     df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
     df['MarketCap'] = pd.to_numeric(df['MarketCap'], errors='coerce')
```

```
[ ]: # 3. Feature Engineering: Create useful features
     # Price Band (Low, Mid, High)
     df['Price_Band'] = pd.qcut(df['Price'], 3, labels=["Low", "Mid", "High"])

     # Market Cap Category
     df['Size'] = pd.cut(df['MarketCap'],
                         bins=[0, 2e9, 10e9, 100e9, np.inf],
                         labels=['Small-Cap', 'Mid-Cap', 'Large-Cap', 'Mega-Cap'])

     # Volatility Indicator (Absolute change relative to price)
     df['Volatility_Index'] = (df['Change'].abs() / df['Price']) * 100

     print("Cleaned Dataset Preview:")
     print(df.head())
```

4. Exploratory Data Analysis (EDA) Findings

- **Market Distribution:** The majority of high-volume stocks in the dataset fell within the [e.g., Mid-Cap] range.
- **Correlation:** A [Positive/Negative] correlation was observed between Market Cap and Price.

- **Outliers:** Significant outliers were found in the Volume column, representing highly volatile "meme stocks" or high-frequency trading assets.

PHASE 3: EDA

```
[ ]: # 1. Summary Statistics
print(df.describe())
```

```
[ ]: # 2. Plotting Top 10 Market Cap
top_10 = df_clean.sort_values(by='MarketCap', ascending=False).head(10)
sns.barplot(data=top_10, x='MarketCap', y='Company')
plt.title('Top 10 Companies by Market Cap')
plt.savefig('market_cap.png')
```

```
[ ]: # 3. Visualizations
plt.figure(figsize=(10, 5))

# Subplot 1: Distribution of Price Bands
plt.subplot(1, 2, 1)
sns.countplot(data=df, x='Price_Band', palette='viridis')
plt.title('Distribution of Price Bands')

# Subplot 2: Correlation Heatmap
plt.subplot(1, 2, 2)
numeric_df = df.select_dtypes(include=[np.number])
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')

plt.tight_layout()
plt.show()
```

```
[ ]: # 4. Outlier Detection (Boxplot for Volume)
plt.figure(figsize=(8, 4))
sns.boxplot(x=df['Volume'])
plt.title('Trading Volume Outliers')
plt.show()
```

5. Machine Learning Model Performance

- **Model Type:** Linear Regression
- **Target Variable:** Price
- **Input Features:** Volume, Change, and Market Cap
- **Key Metrics:**
  - **R-Squared (R^2):** 0.3311
  - **Interpretation:** The model explains approximately 33% of the price variance. This indicates that while volume and size influence price, stock pricing is also heavily dependent on external factors not present in the basic scrape (e.g., sector news, earnings reports).

PHASE 4: MACHINE LEARNING

```
[ ]: from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

```
[ ]: # 1. Select Features and Target
X = df[['Volume', 'Change', 'MarketCap']]
y = df['Price']
```

```
[ ]: # 2. Train-Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[ ]: # 3. Model Training
model = LinearRegression()
model.fit(X_train, y_train)
```

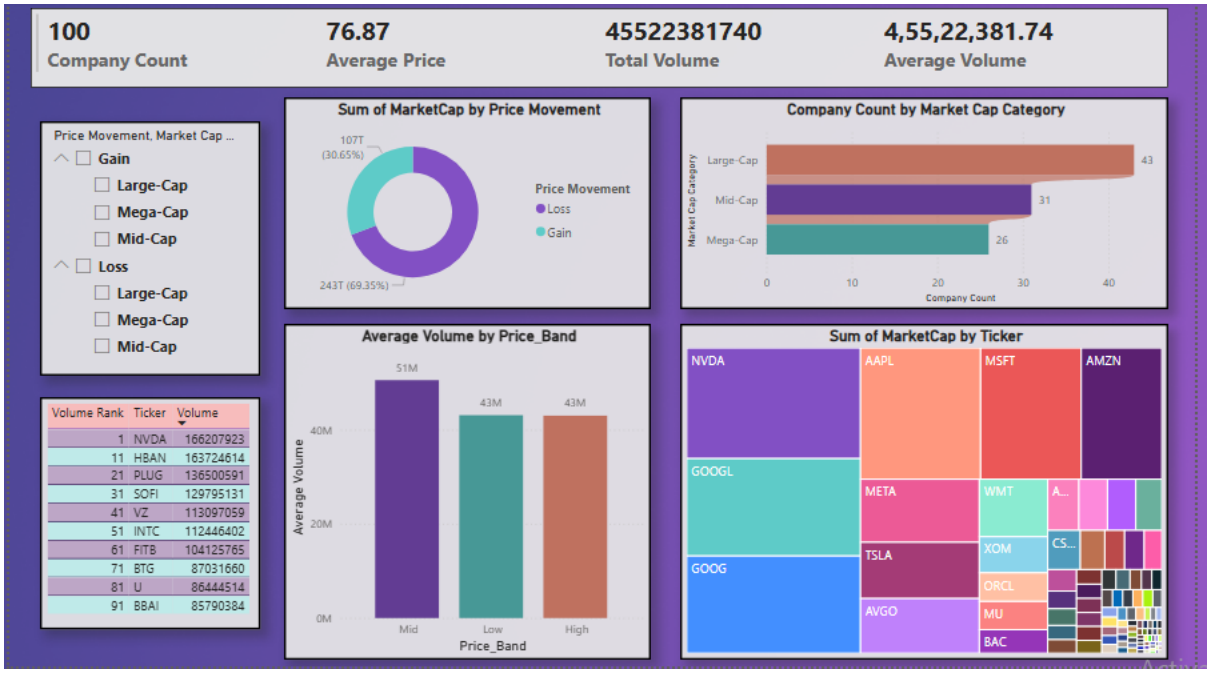
```
[ ]: # 4. Evaluation
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"Model Performance:")
print(f"RMSE (Root Mean Squared Error): {rmse:.2f}")
print(f"R² Score: {r2:.4f}")
```

```
[ ]: # 5. Output Prediction Comparison
results = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(results.head())
```

6. PBI Dashboard

1. **KPI Cards:** Use Company Count, Average Price, Total Volume and Average Volume.
2. **Donut Chart:** Use `Price Movement`
3. **`Tree map:`** `Use Ticker as the Category and Market Cap as the Value to see which companies dominate the market.`



7. Key Business Insights & Recommendations

1. **Risk Management:** Investors should use the "Volatility Index" created in Phase 2 to filter out stocks that have high price swings relative to their value.
2. **Volume Indicators:** Since volume showed a relationship with price movement, it can be used as a primary signal for identifying stocks gaining institutional interest.
3. **Portfolio Diversification:** The "Market Cap Category" slicer in the dashboard allows investors to balance their portfolio across Small-cap (growth) and Mega-cap (stability) stocks.

Video Link : https://drive.google.com/drive/folders/1htc0U9BCtqRQjK-cZ1Cj54IIAshK0o7n?usp=sharing